

✦ JOSA //

Security & the Software Supply Chain

/ Scorecards, Sigstore, SBOMs, and learning from real attacks */*

OpenLab Apprenticeship 2026

Week 07

// 1 //

Learning Goals

- Map the modern OSS supply chain attack surface (typosquatting, dependency confusion, compromised maintainers, malicious updates).
- Use **OpenSSF Scorecard** to evaluate the security health of any GitHub project, including your own.
- Set up Dependabot or Renovate for automated dependency updates.
- Sign a release with **Sigstore (cosign)** and verify a signature.
- Generate an SBOM and read it.
- Discuss the **xz/liblzma backdoor (CVE-2024-3094)** as a case study in maintainer compromise.

// 2 //

Technical Deep Dive

// 2.1 The supply chain in three layers

Modern software is mostly other people's code: an average npm app pulls in 1,000+ transitive dependencies; a Python web app, 200+. Every one of those is an attack surface. Three categories:

- 1. Dependency-on-itself attacks.** A package you depend on is compromised — either a malicious version is published (typosquatting: `colors.js` vs `colours.js`; or a maintainer goes rogue, e.g., `node-ipc`'s "protestware" 2022 incident; or a package is taken over via expired domains or coerced maintainers, e.g., the xz/liblzma backdoor).
- 2. Build-process attacks.** Your CI is compromised. The classic case: a malicious GitHub Action you imported as `@main` (mutable ref) silently changes underneath you. The 2024 tj-actions/changed-files compromise affected hundreds of repos this way.
- 3. Distribution attacks.** Your published artifact is tampered with after build. The defense here is signing — Sigstore.

// 2.2 OpenSSF Scorecard: real, free, immediately useful

Scorecard runs 18+ automated checks on any GitHub repo and produces a 0–10 score. To run it on any public repo:

```
go install github.com/ossf/scorecard/v5/cmd/scorecard@latest
scorecard --repo=github.com/expressjs/express
```

Or visit scorecard.dev to look up pre-computed scores.

Sample checks

- [Branch-Protection](#) — is `main` protected, requiring reviews?
- [Code-Review](#) — do PRs get reviewed before merge?
- [Dangerous-Workflow](#) — does CI run untrusted code as the default branch (`pull_request_target` with checkout)?
- [Pinned-Dependencies](#) — are GitHub Actions pinned by SHA, not tag?
- [Signed-Releases](#) — are releases cryptographically signed?
- [SAST](#) — does the project run static analysis (CodeQL)?
- [Token-Permissions](#) — are GitHub Actions tokens minimally scoped?
- [Vulnerabilities](#) — does osv.dev know of any unpatched CVEs?

// 2.3 Dependabot vs Renovate

Both bot-update your dependencies. Both are free.

- **Dependabot** — built into GitHub, zero-config: just create `.github/dependabot.yml`. Good defaults; opens one PR per outdated dep.
- **Renovate** — third-party (Mend), more configurable. Can group related deps into one PR (`config:base` preset), supports more package managers, has fancier scheduling.



TIP

Default to Dependabot. Switch to Renovate if you want grouping or have a polyglot repo.

A minimal Dependabot config:

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: npm
```

```
directory: /
schedule: { interval: weekly }
open-pull-requests-limit: 10

- package-ecosystem: github-actions
  directory: /
  schedule: { interval: weekly }
```

The [github-actions](#) ecosystem updates your Actions versions – important because pinning to [@v4](#) still gets you any tagged-release vulnerability. Best practice is to pin Actions by **commit SHA**:

```
- uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11 # v4.1.1
```

// 2.4 Sigstore in five minutes

Traditionally, signing software meant managing GPG keys – and almost nobody did it well. Sigstore (a Linux Foundation project; co-founded by Google, Red Hat, Chainguard) makes signing **keyless** by tying signatures to your existing identity (GitHub, Google).

To sign a release artifact with [cosign](#):

```
# Install
brew install cosign

# Sign (opens a browser to authenticate with GitHub)
cosign sign-blob --yes ./release.tar.gz --output-signature release.tar.gz.sig

# The signature is logged to the public Rekor transparency log.
# Anyone can verify:
cosign verify-blob \
  --certificate-identity 'https://github.com/your-org/your-repo/.github/workflows/release.yml@refs/
tags/v1.0.0' \
  --certificate-oidc-issuer 'https://token.actions.githubusercontent.com' \
  --signature release.tar.gz.sig \
  ./release.tar.gz
```

The verifier confirms: *this artifact was signed by a GitHub Actions run on this exact workflow, at this exact tag*. Tampering at any step breaks verification.

// 2.5 SBOM in five minutes

A Software Bill of Materials (SBOM) is a JSON or XML inventory of every dependency in your build. Two main formats: **CycloneDX** (OWASP) and **SPDX** (Linux Foundation). Generate one with [Syft](#):

```
syft ./my-app --output cyclonedx-json=sbom.json
```

The output is a list of every package, version, and license. Vulnerability scanners ([Grype](#), Trivy) consume the SBOM and report known CVEs against [osv.dev](#).



IMPORTANT

US government contractors are now required to ship SBOMs (per Executive Order 14028); the EU Cyber Resilience Act will require similar starting 2027. SBOMs are no longer optional in commercial software.

// 2.6 Case study: the xz backdoor (CVE-2024-3094)

In March 2024, a contributor named “Jia Tan” — who had spent **two years** quietly building maintainer trust on the [xz](#) compression library — landed a backdoor in [liblzma](#) that compromised SSH on most Linux distros. It was caught by accident, by a Microsoft engineer noticing slow SSH logins.

What this teaches

- **A single-maintainer project is a single point of failure.** The original maintainer was burned out and welcomed help; “Jia Tan” had no other identity.
- **Tarball releases are different from git releases.** The malicious code was in the released tarball, not visible in git tags.
- **Long-game social engineering is a real threat model.**

The defenses Scorecard, Sigstore, and OSV are designed against are exactly this threat. Read [Russ Cox’s writeup](#) for the full forensic story.

Learning Resources

Topic	Resource
Health-check tool	OpenSSF Scorecard and github.com/ossf/scorecard
Supply chain framework	SLSA framework (slsa.dev)
Signing	Sigstore (sigstore.dev) and cosign
SBOM tools	Syft + Grype
Vuln database	osv.dev
OWASP Top 10	owasp.org/Top10
Dependency updates	Dependabot / Renovate
The xz backdoor (forensic)	Russ Cox — research.swtch.com/xz-script
Free training	OpenSSF Best Practices for Open Source Developers (LFD121)

Repositories of Interest

- **Any project you've contributed to.**
- [ossf/scorecard](#) itself. Well-organized, documented, friendly to new check contributors.
- [anchore/syft](#) Active SBOM tooling project.

// 5 //

Your Tasks for the Week

1. **Add Dependabot.** Configure Dependabot for one of your repos (with both code and [github-actions](#) ecosystems). Pin all your Actions to commit SHAs. Submit as a PR.
2. **Sign a release.** Tag a release of one of your repos. Sign the artifact with cosign. Document the verify command in your README.
3. **Generate and read an SBOM.** Run Syft on any project. Run Gype against the SBOM. Write a journal entry on what known CVEs (if any) appear, and pick one to investigate (what is it, what's the fix, who's affected).
4. **Run Scorecard on any project,** try to open a PR fixing the lowest-scoring check.

// 6 //

Soft Skill of the Week: Conscientious Skepticism

Security work requires a temperament: you have to be the person who asks “but what if this fails?” without becoming the person nobody invites to the kickoff meeting. The skill is **conscientious skepticism** – the discipline of probing assumptions, but constructively.

// 6.1 Specific Moves

1. **Phrase concerns as questions, not objections.** “What’s the threat model for this dependency?” lands differently from “We can’t add this dependency.”
2. **Bring the data, not the alarm.** Don’t say “this is risky” – say “this dependency has 3 known CVEs in the last year and a Scorecard of 4/10. Here’s the alternative.”
3. **Pick your battles.** Not every Scorecard check matters equally. Branch protection on main is a real fight; Pinned-Dependencies in a hobby project is not.
4. **Default to charity about the team.** If a colleague added a sketchy dependency, they didn’t know – they were trying to ship. Educate, don’t blame.



HEADS UP

There's also a reverse failure mode: **security theater**, where teams adopt every checkbox tool and produce no actual safety. Real security comes from understanding which threats matter to your project and addressing those, not from having the most badges.

// 6.2 Resources

- [OWASP Threat Modeling – Cheat Sheet](#)